

```
#!/usr/bin/env python3
"""
Cybersecurity Threat Detection and Analysis
System
-----
-----
Malware/file-based scanner combining
signature (hash) matching with
heuristic analysis (entropy, suspicious
extensions).

Usage examples:
    python3 main.py --path /some/folder
    python3 main.py --path
suspicious_file.exe
    python3 main.py --path . --no-recursive
    python3 main.py --add-signature
<sha256_hash> "Trojan.Generic" --severity
high
    python3 main.py --path . --export
report.json
    python3 main.py --path . --quiet
"""

import argparse
import sys

from signatures import SignatureDatabase
from scanner import scan_directory
from alerts import AlertManager
```

```

from report import build_report,
print_report, save_report_json

def main():
    parser = argparse.ArgumentParser(
        description="Cybersecurity Threat
Detection and Analysis System (file/malware
scanner)"
    )
    parser.add_argument("--path",
help="File or directory to scan")
    parser.add_argument("--no-recursive",
action="store_true", help="Do not scan
subdirectories")
    parser.add_argument("--quiet",
action="store_true", help="Only print
suspicious/malicious findings")
    parser.add_argument("--export",
metavar="FILE", help="Export full report as
JSON to FILE")
    parser.add_argument(
        "--add-signature", nargs=2,
metavar=("HASH", "NAME"),
        help="Add a known-malicious hash to
the local signature database"
    )
    parser.add_argument("--severity",
default="high", help="Severity for --add-
signature (default: high)")

    args = parser.parse_args()

    sig_db = SignatureDatabase()

```

```
        if args.add_signature:
            file_hash, name =
args.add_signature
            sig_db.add_signature(file_hash,
name, args.severity)
            print(f"Signature added:
{file_hash} -> {name} ({args.severity})")
            if not args.path:
                return

        if not args.path:
            parser.print_help()
            sys.exit(1)

        print(f"Loaded {sig_db.count()} known
threat signatures.")
        print(f"Scanning: {args.path}\n")

        alert_mgr =
AlertManager(quiet=args.quiet)
        results = []

        for result in scan_directory(args.path,
sig_db, recursive=not args.no_recursive):
            alert_mgr.process(result)
            results.append(result)

        report = build_report(args.path,
results, alert_mgr.summary())
        print_report(report)

        if args.export:
            save_report_json(report,
```

```
args.export)
    print(f"Full JSON report saved to:
{args.export}")

if __name__ == "__main__":
    main()
```